Contents

User Research Log — Implementation Plan

Status: v1 plan, ready to schedule **Author:** Joe Pruskowski (with Claude) **Date:** 2026-05-16

Source entry: `doc/Future_Ideas.md` → "Research Log for users — in-platform training journal".

Goal

Move the platform's recommended "keep a research log" workflow **inside** the platform, so that:

1. **Users gain memory.** Every training session can carry structured notes the user can scan a month later.
2. **The platform gains training data.** Each note is paired with the hyperparameters, algorithm, and benchmark/ELO delta of the session that produced it. That triple — *free-form prose + structured config + outcome signal* — is exactly the shape a future reranker or fine-tune wants.
3. **The Help System gets smarter over time.** Published notes (opt-in) become a new corpus source. The retrieval pipeline mixes them in alongside the curated `HelpDoc` chunks, with attribution. Private notes are personal context only.

---

§1 Requirements

1.1 User-facing requirements

ID	Requirement
U1	Every Gym training session can carry **at least one note** (free-form text, ≤2 KB).
U2	Each note has an **outcome label**: `success` / `plateau` / `regression` / `inconclusive`.
U3	Each note has optional **tags** (free-form, comma-separated; normalized to lower-snake).
U4	Notes are **private by default**. A per-note "publish to community" toggle is the only way they become visible to others.
U5	Users can **edit and delete** their own notes. Editing a published note re-runs the PII scrub.
U6	The Gym → Sessions tab shows an inline **"+ Add note"** affordance on every session row. Existing notes are listed under each row, newest first.
U7	A new Profile → **Training journal** tab shows every note the user has written, chronologically, with filters by algorithm / outcome / tag. Markdown rendering. Export to `.md` for backup.
U8	A user can also add **ad-hoc entries** (planning, retrospectives) not tied to a specific session. These live in the Profile journal but never in the Sessions tab.
U9	When the user asks the Guide a training question, the answer can cite their own past notes ("you wrote on 2026-04-12 …"). Citation is gated by a Profile preference (`shareNotesWithGuide`, default ON).

ID	Requirement
U10	When a user opts in to publish a note, the answer can also cite *other users' published notes* — with attribution shown as "Community note from @alice (2026-04-12)".

1.2 Platform / system requirements

ID	Requirement
S1	Schema lives in the existing Prisma model (`packages/db`). Two new tables: `TrainingSessionNote` and `ResearchLogEntry`.
S2	The note layer **does not duplicate** hyperparameters or benchmark data — those already live on the existing `MLSession` row. Notes carry only the editorial content; reads JOIN.
S3	Published notes are **mirrored into the Help corpus** as a new `HelpDoc.source = 'community-note'`. This is an additive change to the existing schema (`HelpDoc.source` is already a free-text label). Reindexing reuses the existing FTS + pgvector pipeline.
S4	The retrieval pipeline returns **three categories** of chunks per query: `corpus` (curated `HelpDocs`), `private-notes` (only the asking user's), `community-notes` (published by others). The mixer assigns each category a budget so curated content always wins ties.
S5	A privacy scrubber runs on every **publish** action: regex over emails / phone numbers / addresses / IP-shaped strings. Display name + user-id are scrubbed; `authorHandle` is set from the user's preferred public handle.
S6	Per-user rate limit on publishing: max 50 published notes / week (matches the existing `helpRateLimit` shape). Editing counts as a new publish for limit purposes.
S7	Notes data is mineable: a nightly job exports `(note, sessionConfig, outcome, eloDelta)` tuples to `ResearchLogExport` (a normalized denormalized view in Postgres) that the reranker / fine-tune pipelines can read directly.

1.3 Out of scope for v1

Post-v1 enhancements (comments, voting, voice input, real-time collaboration, bulk markdown import, per-note hyperparameter hints) are tracked in **`doc/Future_Ideas.md` → "Research Log — post-v1 enhancements"**.

---

§2 Design

2.1 Schema

Two new Prisma models, additive migration. No backfill required — empty tables are fine on first deploy.

```
model TrainingSessionNote {
  id                  String   @id @default(cuid())
  userId              String
  user                User     @relation(fields: [userId], references: [id])
  sessionId           String
  session             MLSession @relation(fields: [sessionId], references: [id])
  parentNoteId        String?
  parent              TrainingSessionNote?  @relation("NoteChain", fields: [parentNoteId], references: [id])
  children            TrainingSessionNote[] @relation("NoteChain")

  body                String   @db.Text           // user-authored markdown, ≤ 2 KB
```

```
  outcome              NoteOutcome
  tags                 String[]                    // normalized lower-snake

  sharedWithCommunity  Boolean  @default(false)
  publishedAt          DateTime?
  helpDocId            String?  @unique             // FK into the mirrored HelpDoc (when published)

  createdAt            DateTime @default(now())
  updatedAt            DateTime @updatedAt

  @@index([userId, createdAt])
  @@index([sessionId])
  @@index([sharedWithCommunity, publishedAt])
}

model ResearchLogEntry {
  id                   String   @id @default(cuid())
  userId               String
  user                 User     @relation(fields: [userId], references: [id])

  title                String                       // ≤ 120 chars
  body                 String   @db.Text            // ≤ 4 KB
  tags                 String[]
  category             ResearchEntryCategory        // PLANNING | RETROSPECTIVE | OBSERVATION | OTHER

  sharedWithCommunity  Boolean  @default(false)
  publishedAt          DateTime?
  helpDocId            String?  @unique

  createdAt            DateTime @default(now())
  updatedAt            DateTime @updatedAt

  @@index([userId, createdAt])
  @@index([sharedWithCommunity, publishedAt])
}

enum NoteOutcome {
  SUCCESS
  PLATEAU
  REGRESSION
  INCONCLUSIVE
}

enum ResearchEntryCategory {
  PLANNING
  RETROSPECTIVE
  OBSERVATION
  OTHER
}
```

Why two tables instead of one: - `TrainingSessionNote` is tightly coupled to a `MLSession` (FK + delete behavior + retrieval-time JOIN to hyperparameters). Optional `sessionId` would muddy that. - `ResearchLogEntry` is standalone — planning posts before a session, retrospectives spanning weeks, observations on bracket UX. Different shape. - Both share the publish-to-corpus pathway; the publish handler dispatches on type but writes to the same `HelpDoc` table.

Why `helpDocId` UNIQUE on the note table: - Mirrors the note's published state in the Help corpus 1:1. Editing a published note updates the linked `HelpDoc`; unpublishing deletes the `HelpDoc` and clears the FK. - Lets the privacy scrubber + chunker rerun idempotently on re-publish.

2.2 API surface

All routes under `/api/v1/research`. Auth required for every route except the public published list (which is also reachable via the Help System retrieval — see §2.4).

Method + Path	Purpose	Notes
`POST /research/sessions/:sessionId/notes`	Create a new note on a training session	Body: `{ body, outcome, tags? }`. Caller must own the session.
`PATCH /research/notes/:noteId`	Edit a note	Body: any of `{ body, outcome, tags }`. Re-runs PII scrub if `sharedWithCommunity`.
`DELETE /research/notes/:noteId`	Delete a note	If published, also deletes the linked `HelpDoc`.
`POST /research/notes/:noteId/publish`	Toggle community-share on	Idempotent. Runs the PII scrubber, creates / updates the `HelpDoc` row, chunks + embeds via the existing Sprint-2 pipeline.
`POST /research/notes/:noteId/unpublish`	Toggle community-share off	Idempotent. Deletes the linked `HelpDoc` (+ its chunks).
`GET /research/notes`	List the caller's notes	Filters: `?algorithm=…&outcome=…&tag=…&since=…&limit=…`. Paginated.
`GET /research/notes/:noteId`	Single note	403 unless owner or note is published.
`POST /research/entries`	Create an ad-hoc `ResearchLogEntry`	Body: `{ title, body, tags?, category }`.
`PATCH /research/entries/:entryId`	Edit	Same publish-rescrub rules.
`DELETE /research/entries/:entryId`	Delete	Same publish-cascade rules.
`POST /research/entries/:entryId/publish`	Publish to community	—
`POST /research/entries/:entryId/unpublish`	Unpublish	—
`GET /research/entries`	List the caller's entries	Filters: `?category=…&tag=…&since=…&limit=…`.
`GET /research/community`	Public list of published notes + entries	Read-only. Surfaces in the Profile journal under a "Community" tab. Rate-limited.
`GET /research/export.md`	One-shot markdown export of all the caller's notes + entries	Server returns `text/markdown` with YAML front-matter per item.

2.3 PII scrubbing on publish

`backend/src/services/research/pii.js` — single pure function `scrubForPublish(text)`:

- Regex strip of: email addresses, phone numbers (loose international match), IPv4 / IPv6, US-style street addresses (rough match — `<num> <Word>+ (Street|Ave|Rd|Blvd|…)`), credit-card-shaped digit runs.
- Heuristic detection of personal names is intentionally **not** attempted in v1 — too high a false-positive rate. The publish flow shows a preview with all matches highlighted so the user can edit before confirming.
- The user's own `displayName` and `username` are preserved if they choose `attributeAs: 'handle'` (the **default**); replaced with `Anonymous` if they explicitly toggle `attributeAs: 'anonymous'` in the publish modal.

UI: the publish modal shows a side-by-side preview (left: original, right: scrubbed). Confirm publishes; Cancel keeps the note private.

2.4 Help System integration

The Help System's retrieval pipeline (Sprint 2 §3 `services/help/retrieval.js`) currently returns top-k chunks from `HelpChunk` filtered by `HelpDoc.publishedAt IS NOT NULL`. Extension:

- **Add a `source` filter** to the retrieval query. Already supported by the schema (`HelpDoc.source` is a free-text label).
- **Three retrieval lanes** mixed at answer-build time:

Lane	`HelpDoc.source` value	Visibility	Default budget
`corpus`	`'guide'` (existing curated docs)	All users	4 chunks

Lane	`HelpDoc.source` value	Visibility	Default budget
`private-notes`	`'private-note'`	Only the asking user	2 chunks
`community-notes`	`'community-note'`	All users	2 chunks

The budget is the **upper bound** per lane — the existing relevance threshold still applies, so a low-quality private note gets cut even when there's room for it.

- **Citation format:** each chunk carries its `HelpDoc.id` + `HelpDoc.source` through the existing retrieval response. The answer renderer (`HelpAnswer.jsx`) gains a small badge: `Guide`, `Your note`, `Community note by @handle`.
- **Privacy filter at write-time, not query-time:** the `private-note` `HelpDoc` rows carry the author's `userId` in `HelpDoc.ownerId` (new optional column, additive migration). The retrieval query adds `WHERE source = 'private-note' AND ownerId = $callerId` for the private lane. Community-note rows have `ownerId IS NULL` (anonymized) or `ownerId = author` (handle-attributed) — either way they're publicly readable.
- **`shareNotesWithGuide` preference:** stored on the user record. When `false`, the `private-notes` lane is skipped entirely for that user's queries. Defaults to `true` because the user has already opted into the platform.

2.5 Data-mining export

Cron job `backend/src/jobs/researchLogExport.js`, runs nightly at 03:00 UTC. Builds a denormalized view-table `ResearchLogExport`:

```
model ResearchLogExport {
  id              String   @id @default(cuid())
  noteId          String?  @unique
  entryId         String?  @unique
  userId          String
  algorithm       String?               // copied from MLSession at export time
  hyperparameters Json?                 // snapshot at session end
  preBenchmark    Float?
  postBenchmark   Float?
  eloBefore       Int?
  eloAfter        Int?
  eloDelta        Int?
  noteText        String   @db.Text
  outcome         NoteOutcome?
  category        ResearchEntryCategory?
  tags            String[]
  publishedAt     DateTime?
  exportedAt      DateTime @default(now())
}
```

Why a denormalized table instead of a query view: training pipelines pull rows in batches; a static table avoids re-JOINing `MLSession` + `BotSkill` + `GameElo` for every batch. The export is rebuilt nightly (truncate + insert) so it's eventually-consistent with the source tables — perfectly fine for offline training jobs.

The reranker (Sprint-5+ in `Future_Ideas.md`) reads from this table directly. Future fine-tune jobs ingest it via `pgcopy`. No new infrastructure needed beyond the cron.

2.6 UI surfaces

Sessions tab (`GymPage.jsx` → Sessions panel):

- Each session row gains an expandable "Notes" drawer (collapsed by default).
- Inside the drawer: existing notes **newest first by default**, with a small sort-toggle button in the drawer header that flips to oldest-first. Each row shows body / outcome pill / tag chips / publish badge / edit + delete affordances.
- "+ Add note" button at the bottom opens an inline editor: textarea, outcome dropdown, tags input (free-form, lower-snake normalized, typeahead from user's own most-used + top community tags).
- Publish button on a saved note opens the side-by-side scrubber preview modal **only when `SystemConfig.researchLog.publishEnabled === true`** (Sprint 2 feature flag — see C6 in §7); otherwise the button is hidden entirely (Sprint 1 behavior).

Profile → Training journal tab (new):

- Section 1: **Session notes** — all `TrainingSessionNote` rows for the caller, filterable by algorithm / outcome / tag / date range. Click a row → expand inline editor or jump to the source session.
- Section 2: **Ad-hoc entries** — all `ResearchLogEntry` rows, filterable by category / tag / date.
- Section 3: **Community feed** — `GET /research/community` paginated. Filter by `tag`. Click a published note → read view. Cross-link to the same feed in the Guide drawer (see below).
- Right-rail: export to `.md` button (calls `GET /research/export.md`).
- Settings cog: `shareNotesWithGuide` toggle.

Guide drawer (`GuidePanel.jsx`):

- Two integration points:
  1. **Citation rendering in `HelpAnswer.jsx`** — the existing citation block gains a small icon prefix per source type and links each citation to its source (Guide doc, your own journal entry, or the published community note).
  2. **New "Community notes" section in the browse panel** — paginated list of published notes alongside the existing `/help/:slug` curated docs. Same `GET /research/community` API as the Profile sub-tab; shared component, two mount points. Cross-link header: "View in Profile journal →" / "View in Guide →".

---

§3 Implementation plan

Three sprints, each independently shippable. Sprint 1 delivers user value standalone; Sprint 2 adds the Profile surface; Sprint 3 wires up Help retrieval.

Sprint 1 — Foundation (Sessions tab + CRUD)

Goal: users can write, edit, and delete notes attached to training sessions, visible inline in Gym → Sessions.

Backend:

1. **Schema** — `TrainingSessionNote` + `NoteOutcome` enum. Prisma migration. Tests: schema round-trip, FK + cascade behavior, unique constraint on `helpDocId`.
2. **Routes** — `POST /research/sessions/:sessionId/notes`, `PATCH /research/notes/:noteId`, `DELETE /research/notes/:noteId`, `GET /research/notes`, `GET /research/notes/:noteId`. All gated by `requireAuth`. Owner check via `req.auth.userId === note.userId`.
3. **Body validation** — `body` ≤ 2 KB, `outcome` must be a `NoteOutcome` value, `tags` array of ≤ 10 lower-snake strings each ≤ 32 chars. Reject with 400 + clear error code.
4. **Tests** — 10 vitest cases on the routes (happy paths + owner check + size limit + outcome validation + missing-session 404).

Frontend (landing):

5. **`api.research.{create,update,delete,list,get}`** in `landing/src/lib/api.js`.
6. **`SessionNotesDrawer.jsx`** — collapsible drawer with notes list + inline editor.
7. **Integration in `GymPage.jsx`** Sessions tab — render `<SessionNotesDrawer sessionId={s.id} />` per row.
8. **Vitest** — 5 cases on the drawer (render, add, edit, delete, optimistic update + rollback on error).

Exit criteria: - A user can add a note to any of their training sessions and see it inline on next render. - A user cannot add a note to another user's session (returns 403 in the API; the button is hidden in the UI). - No publish flow yet — every note is private; the publish button on the row says "(Sprint 2)".

Effort: ~2 dev days.

Sprint 2 — Profile journal + Publish flow

Goal: users can browse / filter / export their notes from the Profile tab and opt-in to publish individual notes to the community.

Backend:

1. **Schema** — `ResearchLogEntry` + `ResearchEntryCategory` enum. `HelpDoc.ownerId` (optional FK to `User`). Migration is additive.
2. **Feature flag** — seed `SystemConfig.researchLog.publishEnabled = false` (default off). All publish-related routes + UI gate on this; the entire Sprint 2 surface ships behind the flag so the publish flow is *built* in Sprint 2 but *activated* only when the flag is flipped (no redeploy needed for activation).
3. **PII scrubber** — `backend/src/services/research/pii.js` with regex set listed in §2.3. Pure function + 15 unit-test cases.

4. **Publish routes** — `POST /research/notes/:noteId/publish`, `POST /research/notes/:noteId/unpublish`, `POST /research/entries/:entryId/publish`, `POST /research/entries/:entryId/unpublish`. Each creates / updates / deletes the linked `HelpDoc` row, runs the existing Sprint-2 chunker + embedder, sets `helpDocId`. Idempotent. Returns 503 when `publishEnabled === false`.
5. **Entry routes** — `POST /research/entries`, `PATCH /research/entries/:entryId`, `DELETE /research/entries/:entryId`, `GET /research/entries`.
6. **Community list** — `GET /research/community` — paginated query over `HelpDoc.source IN ('community-note')` JOIN the source `TrainingSessionNote` / `ResearchLogEntry` for the attribution handle.
7. **Export route** — `GET /research/export.md` returns `text/markdown` with YAML front-matter per item.
8. **Rate limit** — extend `helpRateLimit.js` with a `research-publish` bucket: 50 publishes / week per user.
9. **Tests** — ~25 vitest cases covering PII scrub, publish round-trip, unpublish cascade, community list pagination, export round-trip, rate-limit enforcement.

Frontend (landing):

9. **Profile → Training journal tab** — `landing/src/pages/profile/TrainingJournalTab.jsx`. Filters, list, ad-hoc entry composer, community feed sub-tab.
10. **Publish modal** — side-by-side scrubber preview (`PublishNoteModal.jsx`). Shows highlighted matches + Cancel / Confirm.
11. **Export button** — anchor with `download="training-journal.md"` pointing at `GET /research/export.md`.
12. **Settings toggle** — `shareNotesWithGuide` in the user prefs panel.
13. **Vitest** — ~12 cases on the new components.

Exit criteria: - The Profile tab is fully functional standalone — user can browse, filter, edit, delete, publish, and export their notes. - Published notes appear in `GET /research/community` for any user to browse (both Profile sub-tab and Guide drawer Community section). - The Guide drawer browse panel gets a new "Community notes" section sharing the same component as the Profile sub-tab; the Guide retrieval citations themselves are unchanged (that's Sprint 3). - With `SystemConfig.researchLog.publishEnabled = false`, every publish-related surface (button, modal, community feeds, publish API routes) is hidden / 503s — Sprint 2 ships **dark**, awaiting an explicit flag flip.

Effort: ~3.5 dev days (bumped from 3 to cover the second community-feed mount + feature-flag plumbing).

Sprint 3 — Help retrieval integration

Goal: the Guide cites the user's own notes (and community-published notes) when answering training questions.

Backend:

1. **Retrieval mixer** — refactor `services/help/retrieval.js` to take a `lanes: { corpus: 4, privateNotes: 2, communityNotes: 2 }` parameter and return chunks tagged with their lane. Default lanes wired from `SystemConfig` so we can tune without a deploy.
2. **Private-lane filter** — `WHERE source = 'private-note' AND ownerId = $callerId`. Community-lane filter: `WHERE source = 'community-note'`. Corpus lane: existing default.
3. **Citation metadata** — pass `source` + `ownerId` + linked `noteId` / `entryId` through to the `/help/ask` response so the renderer can show the right badge + deep-link.
4. **Preference gate** — read `user.shareNotesWithGuide`; if false, skip the private lane.
5. **Tests** — ~10 vitest cases: lane budget enforcement, private-lane owner filter, preference gate skip, citation metadata round-trip.

Frontend (landing):

6. **`HelpAnswer.jsx` citation block** — render the source-type badge + deep-link. Hover tooltip: "From your note on 2026-04-12" or "From a community note by @alice".
7. **Vitest** — 3 cases on the renderer (private badge, community badge, attribution scrubbed when `ownerId IS NULL`).

Nightly export job:

8. **`backend/src/jobs/researchLogExport.js`** — runs at 03:00 UTC. Truncates + inserts into `ResearchLogExport`. Vitest covers the JOIN + truncate semantics. Wire into the existing `scheduledJobs.js` dispatcher.
9. **Schema** — `ResearchLogExport` model. Migration additive.

Exit criteria: - Asking the Guide a training question (e.g. "what's a good learning rate for q-learning") returns an answer that cites the user's most-recent relevant note alongside the curated corpus docs. - A user who toggles `shareNotesWithGuide` off sees no private-note citations. - Community notes appear with the correct handle (or "Anonymous" when scrubbed). - The nightly export job populates `ResearchLogExport` with the expected JOIN shape.

Effort: ~3 dev days.

§4 Forward-compatibility — what Help Sprint 5+ inherits

This plan is designed to ship the Research Log **before** the Help System Sprint 5+ work in `Future_Ideas.md`. Sprint 5+ then inherits a corpus, a training-pair table, and a retrieval architecture pre-shaped for its requirements — no separate planning needed for the items listed below.

4.1 What each Sprint 5+ item gets for free

Sprint 5+ item	What Research Log delivers
Reranker (v2)	The `ResearchLogExport` denormalized table is **pre-shaped for offline batch training** — `(prose, hyperparams, outcome, eloDelta, lane, feedback)` tuples ready for `pgcopy`. The 3-lane retrieval mixer in `services/help/retrieval.js` is the natural seam — replace "budget per lane" with "rerank across lanes" once trained, no other refactor needed. Per-lane citation metadata in `HelpFeedback` means thumbs land on lane-tagged sources, giving the reranker lane-mix signal not just per-doc signal.
Embedding upgrade	Roughly **doubles the corpus** with diverse user prose (vs the curated Guide tone). The auto-reindex-on-model-change path from Help Sprint 2 already handles drop-in swaps; community-note chunks reindex via the same pipeline. Larger / more-diverse corpus = easier to measure the quality delta of a new embedding model.
Gap-filling from real traffic	Published community notes **are** gap-filling in a different shape — bottom-up rather than admin-curated. The `/admin/help/metrics` dashboard can grow a "most-cited community notes" panel using existing infrastructure. The curation queue at `/admin/help/queries` handles moderation of community-flagged notes via the same UI.
LoRA fine-tune (v3)	Single biggest unlock. The `ResearchLogExport` rows are literally the training-pair shape: `(prose context, structured params, outcome label)`. The fine-tune can learn what makes an answer HELPFUL in *training contexts* specifically, not just syntactically good. Pre-Research-Log there's no such labelled corpus; post-Research-Log it accrues automatically. The Sprint-5+ "~5k HELPFUL pairs" threshold becomes a function of Research Log adoption rate, not a separate data-collection effort.
Streaming filter / Voice input / Cross-session help history / Per-IP rate limit / PII auto-redact	Orthogonal — no dependency, no benefit. Ship whenever.

4.2 Architectural seams deliberately left open

Each of these is a choice made *in the Research Log plan* specifically so Sprint 5+ slots in cleanly:

1. **3-lane retrieval mixer** (`services/help/retrieval.js`, Sprint 3 §1): the reranker replaces "budget per lane" with "rerank across lanes" by changing only that one function. The API contract above (the `/help/ask` handler) and below (the chunk store) doesn't change.
2. **`HelpDoc.source` discriminator**: already free-text. Adding `'private-note'` and `'community-note'` (Sprint 2) doesn't require schema changes for the reranker; it just adds more rows the reranker scores.
3. **`HelpDoc.ownerId`** (new in Sprint 2): the optional FK enables per-user retrieval beyond Research Log too — Sprint 5+ items like personalized canonical-doc ranking or user-specific corpus subsets get this for free.
4. **`ResearchLogExport` nightly job** (Sprint 3 §8): built for offline consumers. Reranker / fine-tune jobs read from it directly, no new ETL needed. The truncate-and-insert pattern means schema evolution is safe — add columns without backfill, the next nightly run rebuilds.

5. **Per-lane citation metadata in the SSE stream** (Sprint 3 §3): once the answer renderer shows lane badges, feedback on the cited chunks is implicitly lane-tagged. The thumbs-down data the reranker eventually trains on already carries the lane signal.

4.3 What Sprint 5+ still needs fresh planning for

The Research Log doesn't eliminate Sprint 5+ planning — it changes what's left to decide:

- **Embedding-model selection criteria**: corpus characteristics (size, prose diversity, technical-vocabulary density) matter for picking BGE-small vs a larger-dim model. Re-evaluate once the community-note corpus has ~3–6 months of accumulation.
- **LoRA fine-tune compute budget + vendor**: still an open decision (own GPU vs hosted via Together / Anyscale / OpenAI fine-tune API). The data shape is settled; the training-cost / latency / vendor-lock trade-off is not.
- **Streaming filter implementation**: token-level vs chunk-level. Orthogonal to Research Log; same planning effort either way.

---

§5 Sprint checklist

Sprint 1 — Foundation (~2 days)

- ☐ **Schema:** add `TrainingSessionNote` + `NoteOutcome` to `packages/db/prisma/schema.prisma`
- ☐ Generate + apply migration locally (`docker compose run --rm backend npx prisma migrate dev --name research_log_notes`)
- ☐ **Route file:** `backend/src/routes/research.js` mounted at `/research` in `index.js`
- ☐ Implement + test `POST /research/sessions/:sessionId/notes`
- ☐ Implement + test `PATCH /research/notes/:noteId`
- ☐ Implement + test `DELETE /research/notes/:noteId`
- ☐ Implement + test `GET /research/notes` (with `algorithm` / `outcome` / `tag` / `since` / `limit` filters)
- ☐ Implement + test `GET /research/notes/:noteId`
- ☐ Tests: owner check (403 on cross-user mutation), body size limit (≤ 2 KB), invalid outcome (400), missing session (404). Target: 10 vitest cases.
- ☐ **Client API:** `api.research.{createNote, updateNote, deleteNote, listNotes, getNote}` in `landing/src/lib/api.js`
- ☐ **Component:** `landing/src/components/research/SessionNotesDrawer.jsx`
- ☐ Integrate into `landing/src/pages/GymPage.jsx` Sessions tab
- ☐ Vitest: 5 cases on the drawer (render, add, edit, delete, optimistic + rollback)
- ☐ Manual QA: write a note → reload → still there; edit → reflects; delete → gone
- ☐ Update `doc/V1_Acceptance.md` with the new flow

Sprint 2 — Profile journal + Publish flow (~3.5 days)

- ☐ **Schema:** add `ResearchLogEntry` + `ResearchEntryCategory` + `HelpDoc.ownerId`
- ☐ Migration: `research_log_entries_and_publish`
- ☐ **Feature flag:** seed `SystemConfig.researchLog.publishEnabled = false`. Gate every publish route on the flag (return 503 when off). Gate the publish button + Community sub-tab + Guide drawer community section on the flag client-side.
- ☐ **PII scrubber:** `backend/src/services/research/pii.js` + 15 vitest cases
- ☐ **Publish routes** (notes + entries) — wire into existing `services/help/corpusChunker.js` + `embedder.js`
- ☐ **Unpublish cascade** — delete linked `HelpDoc` + chunks; clear `helpDocId`
- ☐ **Entry CRUD routes** (`POST/PATCH/DELETE/GET /research/entries[…]`)
- ☐ **Community list:** `GET /research/community` with attribution JOIN
- ☐ **Export:** `GET /research/export.md` returns markdown with YAML front-matter
- ☐ **Rate limit:** extend `helpRateLimit.js` with `research-publish` bucket (50 / week / user, no daily cap)
- ☐ Tests: 25 vitest cases on publish round-trip, scrub, unpublish cascade, community list, export, rate-limit, **feature-flag gate (publish routes 503 when off, succeed when on)**
- ☐ **Page:** `landing/src/pages/profile/TrainingJournalTab.jsx` — three sections (Session notes / Ad-hoc entries / Community feed); Community section hidden when `publishEnabled === false`
- ☐ **Guide drawer integration:** new "Community notes" section in the browse panel mounting the same `<CommunityFeed>` component as the Profile sub-tab. Cross-link header.
- ☐ **Modal:** `landing/src/components/research/PublishNoteModal.jsx` (side-by-side preview, attribution dropdown defaulting to `handle`)
- ☐ **Settings toggle:** `shareNotesWithGuide` in the existing user-prefs panel
- ☐ Vitest: ~14 cases on the new frontend components (publish flow + feature-flag gate + attribution default + cross-link)
- ☐ Manual QA with flag OFF: publish UI invisible everywhere

- ☐ Manual QA with flag ON: write → publish → see in both Profile + Guide drawer community feeds → unpublish → gone from both but still in your journal
- ☐ Update `doc/Guide_Operations.md` with the publish moderation surface + the `publishEnabled` flag flip procedure

Sprint 3 — Help retrieval integration (~3 days)

- ☐ **Retrieval refactor:** `services/help/retrieval.js` accepts `lanes` param + returns lane-tagged chunks
- ☐ **Default lanes from `SystemConfig`** — `help.lanes.corpus`, `help.lanes.privateNotes`, `help.lanes.communityNotes` (defaults 4 / 2 / 2)
- ☐ **Private-lane owner filter** + preference gate (`shareNotesWithGuide`)
- ☐ **Citation metadata** plumbed through `/help/ask` SSE stream
- ☐ Tests: 10 vitest cases on lanes, owner filter, preference gate, metadata round-trip
- ☐ **Renderer:** badge + deep-link in `landing/src/components/help/HelpAnswer.jsx`
- ☐ Vitest: 3 cases on the citation rendering
- ☐ **Schema:** `ResearchLogExport` + nightly job `backend/src/jobs/researchLogExport.js`
- ☐ Wire into `lib/scheduledJobs.js` dispatcher (03:00 UTC)
- ☐ Tests: 5 vitest cases on the JOIN + truncate semantics
- ☐ Manual QA: ask the Guide "what's a good lr for q-learning" with a relevant private note in scope → citation appears with the right deep-link
- ☐ Update `doc/Help_Corpus/` with a "Community notes" explainer doc (one-pager users can read from the Guide)

---

§6 Risks + mitigations

Risk	Likelihood	Impact	Mitigation
PII slips through the scrubber	Medium	High (privacy)	Side-by-side preview before publish; user must actively confirm. Server-side scrub on every edit. Quarterly review of the regex set against published-note samples.
Community-note corpus gets noisy / spam	Medium	Medium	Per-user rate limit (50 / week). HELP_ADMIN moderation queue (reuse the existing `/admin/help/queries` curation infrastructure) for community notes flagged by feedback thumbs-down.
Private-lane retrieval leaks across users	Low	Critical (privacy)	Owner filter applied at the SQL level, not in app code. Test asserts cross-user query returns zero private chunks. Annual penetration-style review.
Retrieval lane budget drowns curated content	Low	Medium	Curated lane wins ties (default 4 vs 2/2). Reranker (Sprint-5+) re-scores across lanes once trained.
Nightly export grows unbounded	Low	Low	`ResearchLogExport` is a truncate-and-insert view — its size is bounded by source tables. Notes themselves have row-level TTL governance via the existing `pruneIfNeeded` job.

---

§7 Decisions locked (2026-05-16 walkthrough)

All v1 decisions confirmed in a planning walkthrough with the project lead. Snapshot below for future readers.

Privacy & security

#	Decision	Locked
A1	`shareNotesWithGuide` preference default	**ON** — private notes mix into Help retrieval by default; toggle in Profile settings to disable
A2	Community-note moderation model	**post-hoc takedown** — notes go live immediately; `/admin/help/queries` curation queue serves as reactive review; 50/week rate limit + thumbs-down feedback flag spam
A3	PII scrubber — personal-name detection	**skip in v1** — emails / phones / IPs / addresses / cards stripped; names left to user via side-by-side preview before confirm
A4	Attribution default on publish	**`handle`** — user's public handle by default; `anonymous` is opt-in via the publish modal dropdown

UX

#	Decision	Locked
B1	Schema split for notes vs entries	**two tables** — `TrainingSessionNote` (FK to `MLSession`) + `ResearchLogEntry` (standalone)
B2	Community-feed location	**both surfaces** — Profile → Training Journal → Community sub-tab AND a new "Community notes" section in the Guide drawer browse panel; shared `GET /research/community` API
B3	Note ordering on a session row	**newest first by default**, with a per-tab sort toggle in the drawer header
B4	Post-v1 enhancements (comments, voting, voice, real-time collab, bulk import, hyperparameter hints)	**deferred — tracked in `doc/Future_Ideas.md` → "Research Log — post-v1 enhancements"**

Implementation details

#	Decision	Locked
C1	Markdown rendering library	**`react-markdown`** — reused from Help System; supports code blocks + GFM tables
C2	Tag taxonomy	**free-form** — normalized to lower-snake-case + dedup; typeahead of user's own most-used + top community tags
C3	Published-then-deleted note behavior	**hard cascade** — `HelpDoc` + chunks deleted; historical citations render as "(note removed by author)"
C4	Body size limits	**2 KB notes / 4 KB entries**
C5	Publish rate limit	**50 / week per user** (no daily cap); reuses the `helpRateLimit.js` shape

#	Decision	Locked
C6	Sprint sequencing	**Sprint 1 ships standalone with publishing hidden**. Sprint 2 builds the publish flow behind `SystemConfig.researchLog.publishEnabled` — when the flag flips, the publish button appears; flag flip is the activation, not a redeploy
C7	Help retrieval lane budgets	**4 corpus / 2 private-notes / 2 community-notes, curated wins ties**; all readable from `SystemConfig` (`help.lanes.corpus`, `help.lanes.privateNotes`, `help.lanes.communityNotes`) so they tune without a deploy

Sequencing

#	Decision	Locked
D1	Research Log vs Help System Sprint 5+ ordering	**Research Log first.** Sprint 5+ items (reranker, embedding upgrade, gap-filling, LoRA fine-tune) all inherit corpus + training-pair data from the Research Log work — see §4 for inheritance details

---

§8 Cross-references

- `doc/Future_Ideas.md` — Research Log entry (this plan's source)
- `archive/Help_System_Plan.md` — retrieval pipeline, chunker, embedder (Sprint 2 §3)
- `archive/Help_System_Sprint_Tracker.md` — corpus + admin curation patterns to mirror
- `doc/V1_Acceptance.md` — QA flows to extend with Research Log paths once Sprints 1 + 2 ship
- `doc/Observability_Plan.md` — metrics to add (`researchNotesCreated`, `researchNotesPublished`, `researchCitationsServed`)